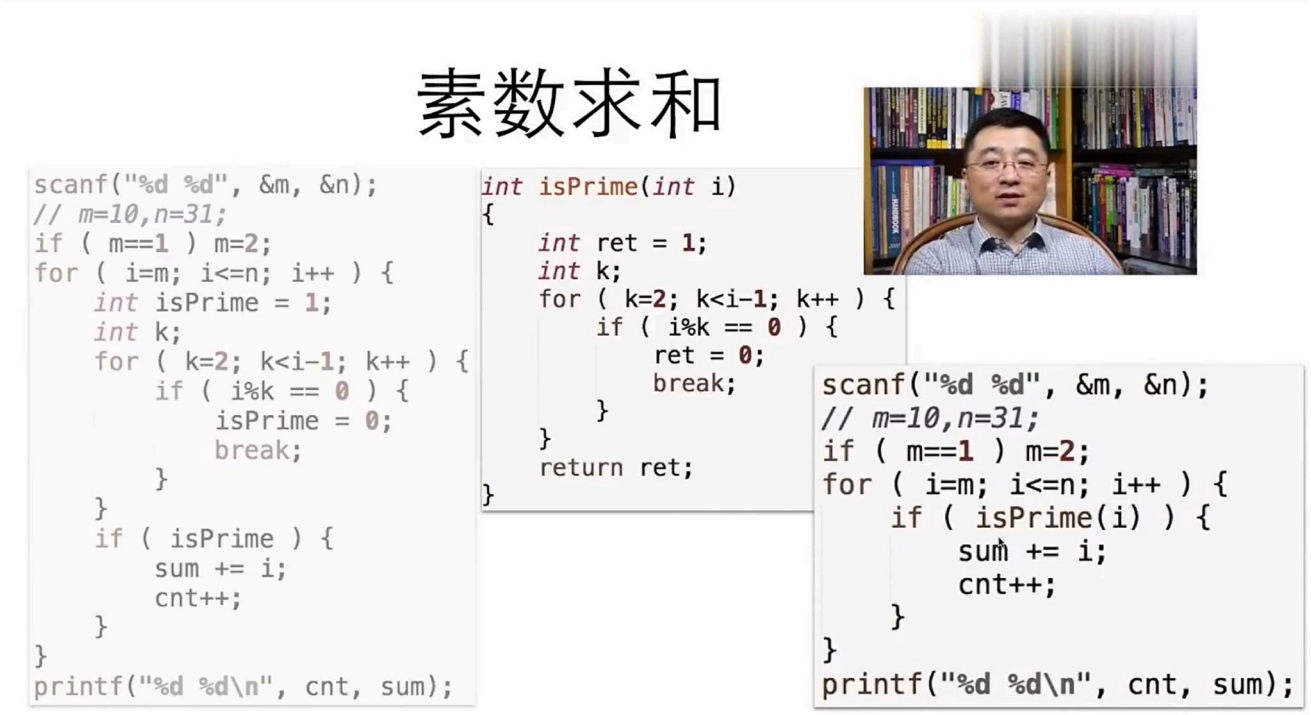
3 C语言核心

对应课程序号:7函数、8数组、9指针、10字符串

46初见函数



求和

- 求出1到10、20到30和35到45的三个和

求和



```
int i;
int sum;

for ( i=1,sum=0; i<=10; i++ ) {
    sum += i;
}
printf("%d到%d的和是%d\n"

for ( i=20,sum=0; i<=30; i++ ) {
    sum += i;
}
printf("%d到%d的和是%d\n", 20, 30, sum);

for ( i=35,sum=0; i<=45; i++ ) {
    sum += i;
}
printf("%d到%d的和是%d\n", 35, 45, sum);
```

“代码复制”是程序质量不良的表现

- 三段几乎一模一样的代码!

求和函数

```
int i;
int sum;

for ( i=1,sum=0; i<=10; i++ ) {
    sum += i;
}
printf("%d到%d的和是%d\n", 1, 10, s

for ( i=20,sum=0; i<=30; i++ ) {
    sum += i;
}
printf("%d到%d的和是%d\n", 20, 30,

for ( i=35,sum=0; i<=45; i++ ) {
    sum += i;
}
printf("%d到%d的和是%d\n", 35, 45,
```

```
void sum(int begin, int end)
{
    int i;
    int sum =0;
    for ( i=begin; i<=end; i++ ) {
        sum += i;
    }
    printf("%d到%d的和是%d\n", begin, end, sum);
}

int main()
{
    sum(1,10);
    sum(20,30);
    sum(35,45);

    return 0;
}
```

```
1   #define _CRT_SECURE_NO_WARNINGS 1
2   #include <stdio.h>
3
4   // 定义求和函数，void 表示这个函数不返回结果，直接在内部打印
5   void sum(int begin, int end)// 函数名叫 sum
6   {
7       int i;
8       int total = 0;// 变量名叫 total
9       for (i = begin;i <= end;i++) {
10          total += i;
11      }
12      printf("%d到%d的和是%d\n", begin, end, total);
```

```
}

int main()
{
    sum(1, 10);
    sum(20, 30);
    sum(35, 45);

    return 0;
}
```

47函数定义和调用

什么是函数?

- 函数是一块代码，接收零个或多个参数，做一件事情，并返回零个或一个值
- 可以先想像成数学中的函数: y=f(x)

函数定义



返回类型

函数名

参数表

函数头

函数体

```
void sum(int begin, int end)
{
    int i;
    int sum =0;
    for ( i=begin; i<=end; i++ ) {
        sum += i;
    }
    printf("%d到%d的和是%d\n", begin, end, sum);
}
```

调用函数

- 函数名(参数值);
- ()起到了表示函数调用的重要作用
  - 即使没有参数也需要()
- 如果有参数，则需要给出正确的数量和顺序
- 这些值会被按照顺序依次用来初始化函数中的参数

```
sum(1,10);
sum(20,30);
sum(35,45);
```

```
void sum(int begin, int end)
{
    int i;
    int sum =0;
    for ( i=begin; i<=end; i++ ) {
        sum += i;
    }
    printf("%d到%d的和是%d\n", begin, end, sum);
}
```

函数返回

- 函数知道每一次是哪里调用它，会返回到正确的地方

```
void sum(int begin, int end)
{
    int i;
    int sum =0;
    for ( i=begin; i<=end; i++ ) {
        sum += i;
    }
    printf("%d到%d的和是%d\n", begin, end, sum);
}
```

```
sum(1,10);
sum(20,30);
sum(35,45);

return 0;
```

48从函数中返回

素数求和

```c
scanf("%d %d", &m, &n);
// m=10,n=31;
if ( m==1 ) m=2;
for ( i=m; i<=n; i++ ) {
    int isPrime = 1;
    int k;
    for ( k=2; k<i-1; k++ ) {
        if ( i%k == 0 ) {
            isPrime = 0;
            break;
        }
    }
    if ( isPrime ) {
        sum += i;
        cnt++;
    }
}
printf("%d %d\n", cnt, sum);
```

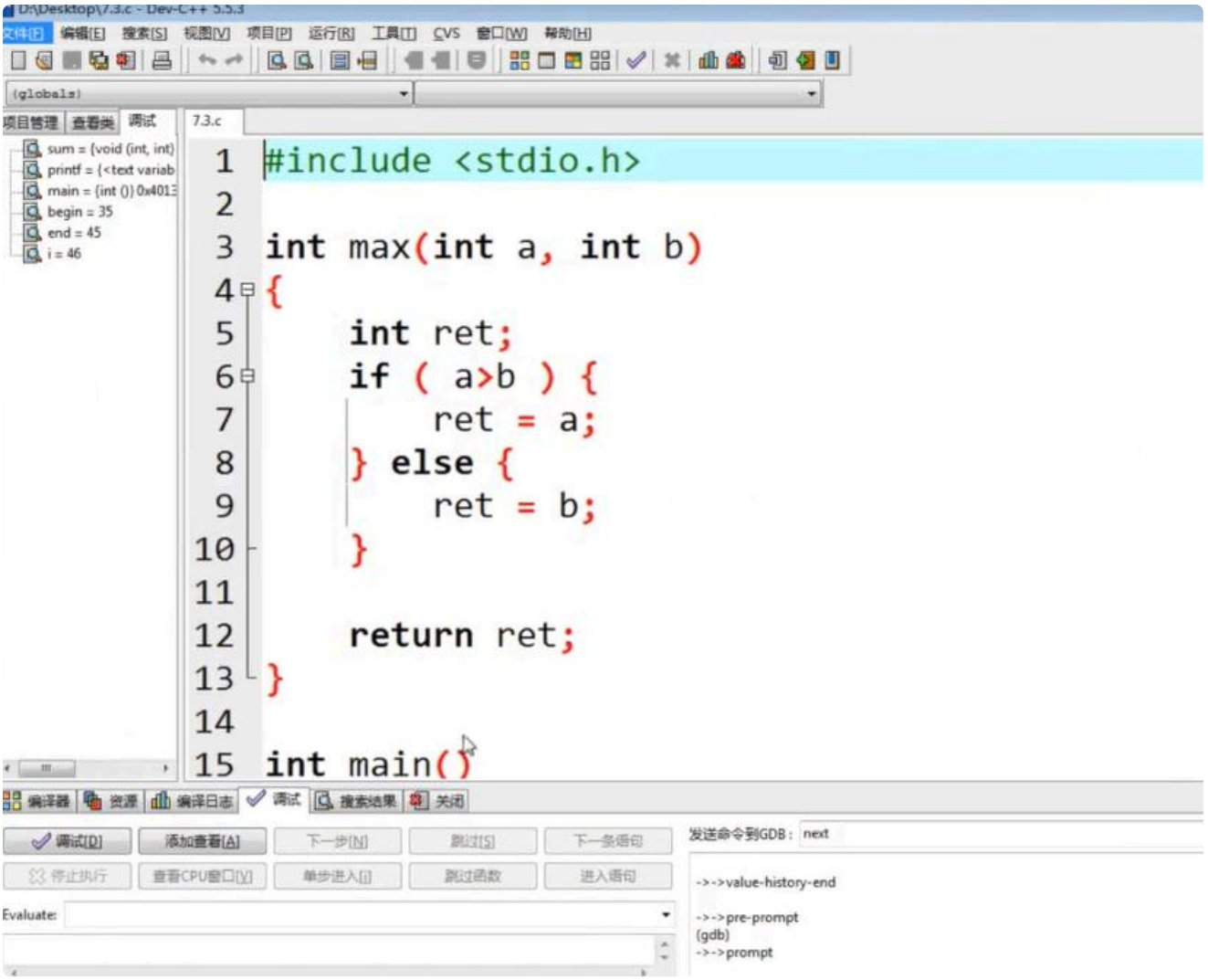
```c
int isPrime(int i)
{
    int ret = 1;
    int k;
    for ( k=2; k<i-1; k++ ) {
        if ( i%k == 0 ) {
            ret = 0;
            break;
        }
    }
    return ret;
}
```

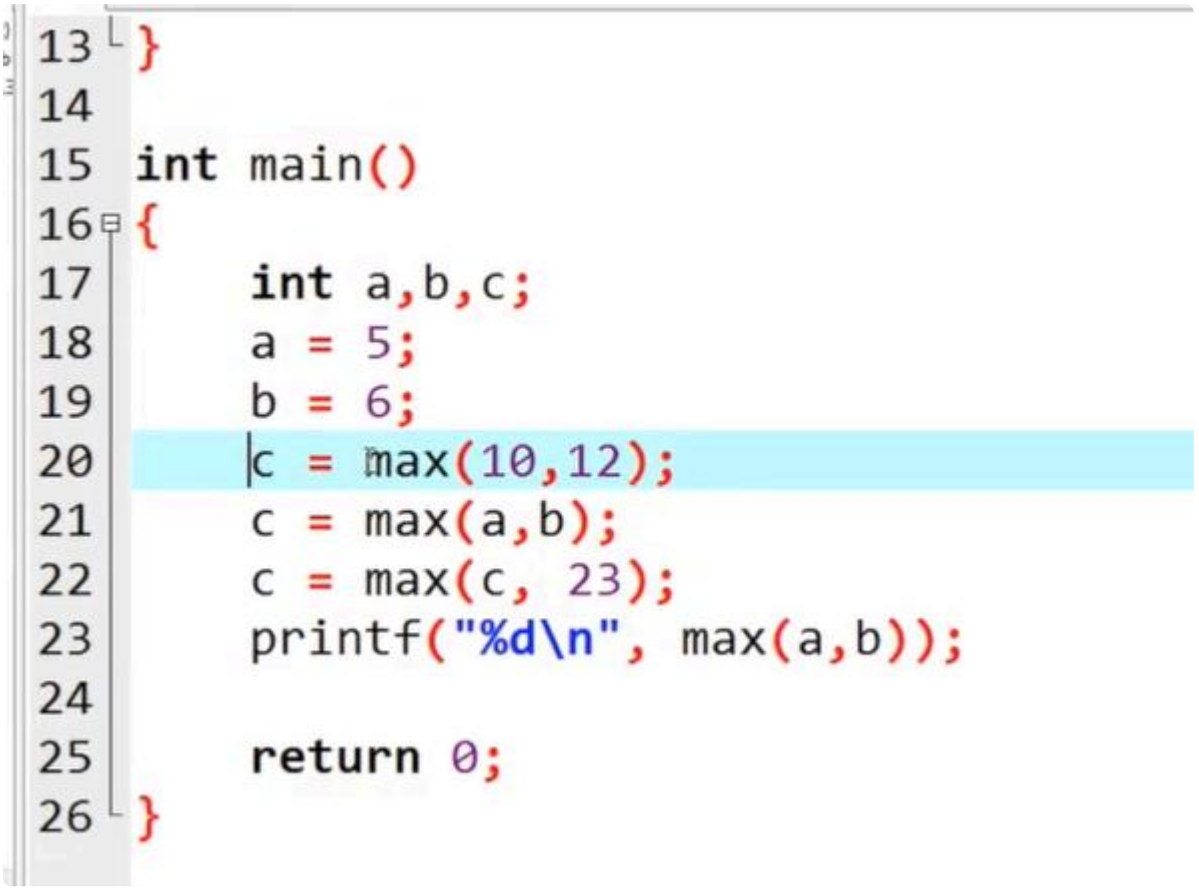
从函数中返回值

```c
int max(int a, int b)
{
    int ret;
    if ( a>b ) {
        ret = a;
    } else {
        ret = b;
    }

    return ret;
}
```

- return停止函数的执行，并送回一个值
  - return;
  - return 表达式;
- 一个函数里可以出现多个return语句


D:\Desktop\7.3.c - Dev-C++ 5.5.3
文件[F] 编辑[E] 搜索[S] 视图[V] 项目[P] 运行[R] 工具[T] CVS 窗口[W] 帮助[H]
(globals)
项目管理 查看类 调试
7.3.c
sum = {void (int, int)
printf = {<text variab
main = {int ()} 0x401
begin = 35
end = 45
i = 46
1 #include <stdio.h>
2
3 int max(int a, int b)
4 {
5 int ret;
6 if (a>b) {
7 ret = a;
8 } else {
9 ret = b;
10 }
11
12 return ret;
13 }
14
15 int main()
编译器 资源 编译日志 调试 搜索结果 关闭
调试[D] 添加查看[A] 下一步[N] 跳过[S] 下一条语句
停止执行 查看CPU窗口[V] 单步进入[i] 跳过函数 进入语句
Evaluate:
发送命令到GDB: next
->->value-history-end
->->pre-prompt
(gdb)
->->prompt


13 }
14
15 int main()
16 {
17 int a,b,c;
18 a = 5;
19 b = 6;
20 c = max(10,12);
21 c = max(a,b);
22 c = max(c, 23);
23 printf("%d\n", max(a,b));
24
25 return 0;
26 }

从函数中返回值

```c
int max(int a, int b)
{
    int ret;
    if ( a>b ) {
        ret = a;
    } else {
        ret = b;
    }

    return ret;
}
```

```c
int a,b,c;
a = 5;
b = 6;
c = max(10,12);
c = max(a,b);
c = max(c, 23);
c = max(max(c,a), 5);
printf("%d\n", max(a,b));
max(12,13);
```

- 可以赋值给变量
- 可以再传递给函数
- 甚至可以丢弃

没有返回值的函数

- void 函数名(参数表)
- 不能使用带值的return
  - 可以没有return
- 调用的时候不能做返回值的赋值

```c
void sum(int begin, int end)
{
    int i;
    int sum =0;
    for ( i=begin; i<=end; i++ ) {
        sum += i;
    }
    printf("%d到%d的和是%d\n", begin, end, sum);
}
```

如果函数有返回值，则必须使用带值的return

49函数原型

函数先后关系

```c
void sum(int begin, int end)
{
    int i;
    int sum =0;
    for ( i=begin; i<=end; i++ ) {
        sum += i;
    }
    printf("%d到%d的和是%d\n", begin, end, sum);
}

int main()
{
    sum(1,10);
    sum(20,30);
    sum(35,45);

    return 0;
}
```

- 像这样把sum()写在上面，是因为：
- C的编译器自上而下顺序分析你的代码
- 在看到sum(1,10)的时候，它需要知道sum()的样子
- 也就是sum()要几个参数，每个参数的类型如何，返回什么类型
- 这样它才能检查你对sum()的调用是否正确

```c
#include <stdio.h>

void sum(int begin, int end);

int main()
{
    sum(1,10);  //  int sum(int,int)
    sum(20,30);
    sum(35,45);

    return 0;
}

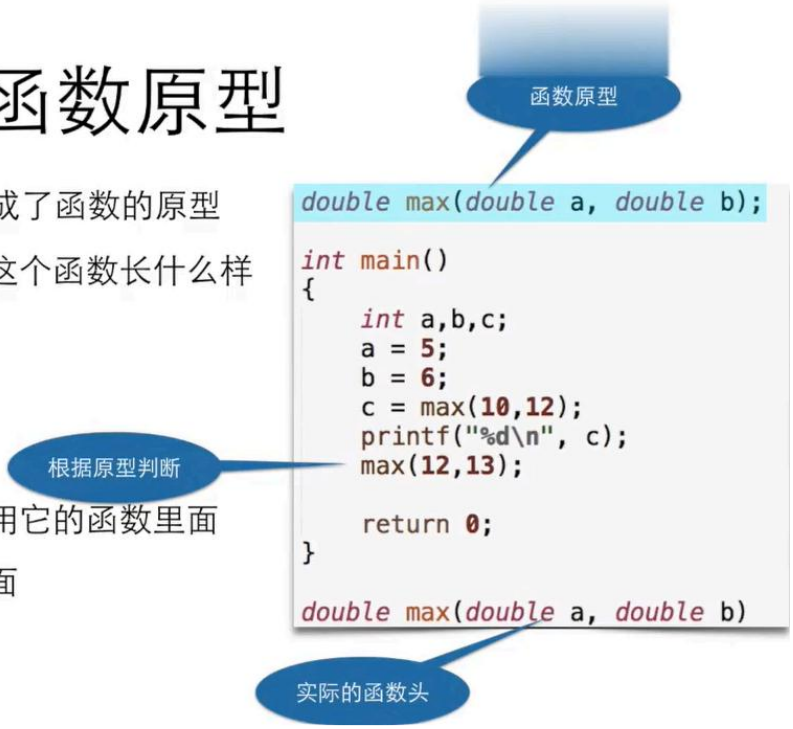
void sum(int begin, int end)
{
    int i;
    int sum =0;
    for ( i=begin; i<=end; i++ ) {
        sum += i;
    }
    printf("%d到%d的和是%d\n", begin, end, sum);
}
```

原型声明

```
[Finished in 0.2s]
```

函数原型

- 函数头，以分号“;”结尾，就构成了函数的原型
- 函数原型的目的是告诉编译器这个函数长什么样
  - 名称
  - 参数（数量及类型）
  - 返回类型
- 旧标准习惯把函数原型写在调用它的函数里面
- 现在一般写在调用它的函数前面



50.参数传递

调用函数



- 如果函数有参数，调用函数时必须传递给它数量、类型正确的值
- 可以传递给函数的值是表达式的结果，这包括：
  - 字面量
  - 变量
  - 函数的返回值
  - 计算的结果

```
int a,b,c;
a = 5;
b = 6;
c = max(10,12);
c = max(a,b);
c = max(c, 23);
c = max(max(23,45), a);
c = max(23+45, b);
```

类型不匹配?

- 调用函数时给的值与参数的类型不匹配是C语言传统上最大的漏洞
- 编译器总是悄悄替你把类型转换好，但是这很可能不是你所期望的
- 后续的语言，C++/Java在这方面很严格

传过去的是什么?

```c
void swap(int a, int b);

int main()
{
    int a = 5;
    int b = 6;

    swap(a,b);

    printf("a=%d b=%d\n", a, b);

    return 0;
}

void swap(int a, int b)
{
    int t = a;
    a = b;
    b = t;
}
```

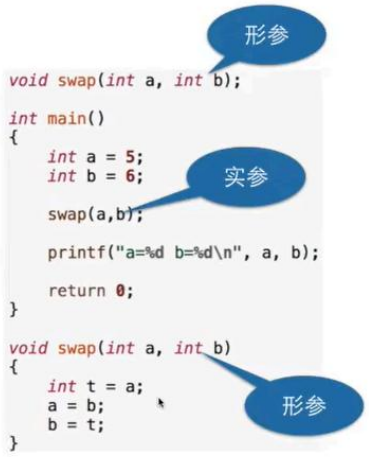
C语言在调用函数时，永远只能传值给函数

- 这样的代码能交换a和b的值吗?

不能!

传值

- 每个函数有自己的变量空间，参数也位于这个独立的空间中，和其他函数没有关系
- 过去，对于函数参数表中的参数，叫做“形式参数”，调用函数时给的值，叫做“实际参数”
- 由于容易让初学者误会实际参数就是实际在函数中进行计算的参数，误会调用函数的时候把变量而不是值传进去了，所以我们不建议继续用这种古老的方式来称呼它们




```
void swap(int a, int b);

int main()
{
    int a = 5;
    int b = 6;

    swap(a,b);

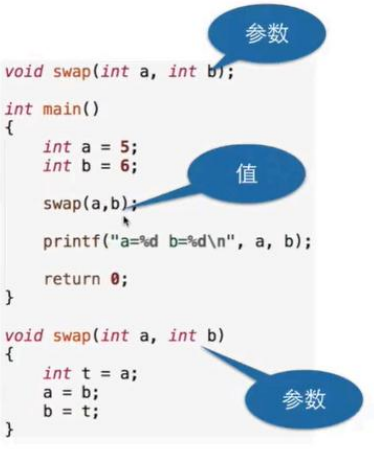
    printf("a=%d b=%d\n", a, b);

    return 0;
}

void swap(int a, int b)
{
    int t = a;
    a = b;
    b = t;
}
```

传值

- 每个函数有自己的变量空间，参数也位于这个独立的空间中，和其他函数没有关系
- 过去，对于函数参数表中的参数，叫做“形式参数”，调用函数时给的值，叫做“实际参数”
- 由于容易让初学者误会实际参数就是实际在函数中进行计算的参数，误会调用函数的时候把变量而不是值传进去了，所以我们不建议继续用这种古老的方式来称呼它们
- 我们认为，它们是参数和值的关系




```
void swap(int a, int b);

int main()
{
    int a = 5;
    int b = 6;

    swap(a,b);

    printf("a=%d b=%d\n", a, b);

    return 0;
}

void swap(int a, int b)
{
    int t = a;
    a = b;
    b = t;
}
```

51.本地变量

本地变量



- 函数的每次运行，就产生了一个独立的变量空间，在这个空间中的变量，是函数的这次运行所独有的，称作本地变量
- 定义在函数内部的变量就是本地变量
- 参数也是本地变量

变量的生存期和作用域



- 生存期：什么时候这个变量开始出现了，到什么时候它消亡了
- 作用域：在（代码的）什么范围内可以访问这个变量（这个变量可以起作用）
- 对于本地变量，这两个问题的答案是统一的：大括号内——块

```c
#include <stdio.h>

void swap(int a, int b);

int main()
{
    int a = 5;
    int b = 6;

    swap(a,b);

    printf("a=%d,b=%d\n", a, b);

    return 0;
}
```

```c

void swap(int x, int y)
{
    int swap;
    int t = x;
    x = y;
    y = t;
}
```

本地变量的规则

- 本地变量是定义在块内的
  - 它可以是定义在函数的块内
  - 也可以定义在语句的块内
  - 甚至可以随便拉一对大括号来定义变量
- 程序运行进入这个块之前，其中的变量不存在，离开这个块，其中的变量就消失了
- 块外面定义的变量在里面仍然有效
- 块里面定义了和外面同名的变量则掩盖了外面的
- 不能在一个块内定义同名的变量
- 本地变量不会被默认初始化
- 参数在进入函数的时候被初始化了

52.函数庶事

其他细节

没有参数时

- void f(void);
- 还是
- void f();
  - 在传统C中，它表示f函数的参数表未知，并不表示没有参数

ret 0???

为什么会这样？因为你在 `main` 里传的是 `int` 类型的 `a=5, b=6`，但函数 `swap` 却非要把这两个数据当成 `double`（8个字节）去解析！内存全部错位了，读出了一堆天文数字（垃圾值）。

🎯 考研（408）必考的核心考点：

这在《计算机组成原理》和 C 语言中叫**"类型不匹配引发的未定义行为（Undefined Behavior）"**。

直接抄进考研笔记：

1. **函数声明时，如果没有任何参数，必须写 `void`（即 `void f(void)`）。**
2. 绝对不要写 `void f()`。这会给编译器留下漏洞，导致你传错了参数类型，它也不报错，最后查出 Bug 找到吐血。
3. **函数声明和定义的参数类型必须完全一致！**（你第 5 行声明的是 `void`，第 25 行定义的是 `double`，这在正经项目里绝对会被开除）。

去 VS 里把第 5 行的 `void swap(void);` 删掉或者改对，把第 12 行的传参删掉。以后写代码，只要函数没参数，**死死记住带上 `void`**。这就是高分选手和踩坑选手的区

继续往下推 7.2.2（参数传递），那是你学指针前的最后一道防线！



逗号运算符?

- 调用函数时的逗号和逗号运算符怎么区分?
- 调用函数时的圆括号里的逗号是标点符号，不是运算符
  - f(a,b)
  - f((a,b))

在调用函数的时候，括号里面的逗号是标志符号，而不是运算符。但是如果在里面在加一层圆括号，就表明要先做里面的运算。

函数里的函数?

- C语言不允许函数嵌套定义

可以在函数里面放另一个函数的声明，但不能放定义。

这是什么?

- int i,j,sum(int a, int b);
- return (i);

不要这样写，没好处

关于main

- int main()也是一个函数
- 要不要写成int main(void)?
- return的0有人看吗?
  - Windows：if errorlevel 1 …
  - Unix Bash：echo $?
  - Csh：echo $status

gcc是什么

53.初试数组

• 如何写一个程序计算用户输入的数字的平均数?

```
int x;
double sum = 0;
int cnt = 0;
scanf("%d", &x);
while ( x!= -1 ) {
    sum += x;
    cnt ++;
    scanf("%d", &x);
}
if ( cnt >0 ) {
    printf("%f\n", sum/cnt);
}
```

- 不需要记录输入的每一个数

• 如何写一个程序计算用户输入的数字的平均数，并输出所有大于平均数的数?

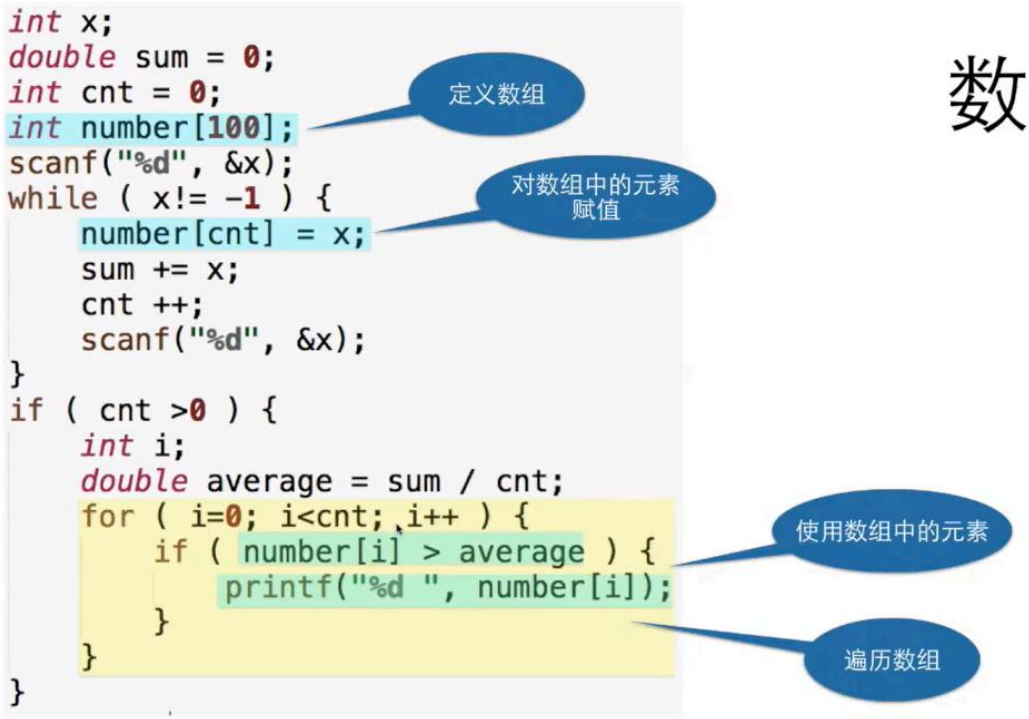
8.1-r.c

```c
#include <stdio.h>

int main()
{
    int x;
    double sum = 0;
    int cnt = 0;
    int number[100];
    scanf("%d", &x);
    while ( x!= -1 ) {
        number[cnt] = x;
        //
        {
            int i;
            printf("%d\t", cnt);
            for ( i=0; i<=cnt; i++ ) {
                printf("%d\t", number[i]);
            }
            printf("\n");
        }
        //
        sum += x;
        cnt ++;
        scanf("%d", &x);
    }
    if ( cnt >0 ) {
        printf("%f\n", sum/cnt);
        int i;
        for ( i=0; i<cnt; i++ ){
            if ( number[i] > sum/cnt ) {
                printf("%d\n", number[i]);
```

```c
        }
    }
}
    return 0;
}
```

数组


int x;
double sum = 0;
int cnt = 0;
int number[100];
scanf("%d", &x);
while (x!= -1) {
number[cnt] = x;
sum += x;
cnt ++;
scanf("%d", &x);
}
if (cnt >0) {
int i;
double average = sum / cnt;
for (i=0; i<cnt; i++) {
if (number[i] > average) {
printf("%d ", number[i]);
}
}
}
定义数组
对数组中的元素赋值
使用数组中的元素
遍历数组

🚨 C语言安全隐患：数组越界 (Buffer Overflow)

1. 隐患现象

使用固定大小的数组接收不确定数量的用户输入时，导致程序崩溃、死循环或产生未知的乱码。

2. 致命代码复现

c 复制 下载

```c
int number[100]; // 数组容量只有 100（合法下标 0~99）
int cnt = 0;
int x;
scanf("%d", &x);
while ( x != -1 ) {
    number[cnt] = x; // 🚨 如果 cnt 达到 100，这里就会越界写入！
    cnt++;
    scanf("%d", &x);
}
```

3. 为什么会发生?

- C 语言**不检查数组边界**。它只认内存地址，`number[100]` 会被编译器强行解释为数组后面的那块内存。
- 如果一直不输入 `-1`，`cnt` 就会无限增长。一旦 `cnt >= 100`，程序就会把数据强行覆盖到其他变量（比如 `sum` 或 `cnt` 本身）的内存上。
- **后果**：数据被无声篡改、程序直接崩溃报 **段错误 (Segmentation Fault)**。在企业开发中，这是黑客最喜欢利用的安全漏洞。

4. 🛡️ 防御性编程（修复方案）

必须加一层"安全阀"：

c 复制 下载

```
while ( x != -1 ) {
    if (cnt < 100) {  // ✅ 防御性检查：满了就不装了
        number[cnt] = x;
        sum += x;
        cnt++;
    } else {
        printf("数组已满，停止录入！\n");
        break; // 强行跳出循环
    }
    scanf("%d", &x);
}
```

5. 🎯 考研 408 视角的升华

- **C语言的痛点**：静态数组（如 `int a[100]`）大小是固定的，开小了越界，开大了浪费内存。
- **数据结构的意义**：这直接引出了王道《数据结构》第二章的核心——**动态分配。**
  - 为了不浪费、不越界，我们可以用指针 `malloc` 动态申请内存（可变数组）。
  - 或者直接用**链表**，来一个数申请一个节点，完美解决容量限制问题。

💡 附：输入死循环隐患（加分项）

- 如果用户手滑输入了字母（如 `'a'`），`scanf("%d", &x)` 读取失败，`x` 的值不会改变，永远是上一次的值，导致 `while(x != -1)` 永远成立，程序彻底死循环。
- **修复**：`while (scanf("%d", &x) == 1 && x != -1)`（检查返回值，确保成功读入了1个整数）。

去把这段直接贴进你的笔记里，以后无论是 PTA 刷题还是考研手写代码，看到数组，条件反射先在脑子里弹出"防越界"这三个字！继续冲！



scanf是什么意思

54定义数组

1.定义数组
• <类型>变量名称`[元素数量]`;
• int grades`[100]`;
• double weight`[20]`;
• 元素数量必须是整数
• C99之前:元素数量必须是编译时刻确定的字面量

2.数组
• 是一种容器(放东西的东西)，特点是:
• 其中所有的元素具有相同的数据类型;
• 一旦创建，不能改变大小
• `*`(数组中的元素在内存中是连续依次排列的)

3.int `a[10]`
• 一个int的数组
• 10个单元:`a[0],a[1].....a[9]`
• 每个单元就是一个int类型的变量
• 可以出现在赋值的左边或右边:
• `a[2]=a[1]+6`;
• `*`在赋值左边的叫做左值

4.数组的单元
• 数组的每个单元就是数组类型的一个变量使用数组时放在[]中的数字叫做下标或索引，下标从0开始计数:
• grades[o]、grades[99]、average[5]

5.有效的下标范围
• 编译器和运行环境都不会检查数组下标是否越界，无论是对数组单元做读还是写
• 一旦程序运行，越界的数组访问可能造成问题，导致程序崩溃，例segmentation fault
• 但是也可能运气好，没造成严重的后果
• 所以这是程序员的责任来保证程序只使用有效的下标值:`[0,数组的大小-1]`

```c
int x;
double sum = 0;
int cnt = 0;
int number[100];
scanf("%d", &x);
while ( x!= -1 ) {
    number[cnt] = x;
    sum += x;
    cnt ++;
    scanf("%d", &x);
}
if ( cnt >0 ) {
    int i;
    double average = sum / cnt;
    for ( i=0; i<cnt; i++ ) {
        if ( number[i] > average ) {
            printf("%d ", number[i]);
        }
    }
}
```



- 这个程序是危险的，因为输入的数据可能超过100个



计算平均数

- 如果先让用户输入有多少数字要计算，可以用C99的新功能来实现

```c
int x;
double sum = 0;
int cnt;
printf("请输入数字的数量: ");
scanf("%d", &cnt);
if ( cnt>0 ) {
    int number[cnt];
    scanf("%d", &x);
    while ( x!= -1 ) {
        number[cnt] = x;
        sum += x;
        cnt ++;
        scanf("%d", &x);
    }
```

C99 ONLY!

长度为0的数组?

- int a[0];
- 可以存在，但是无用

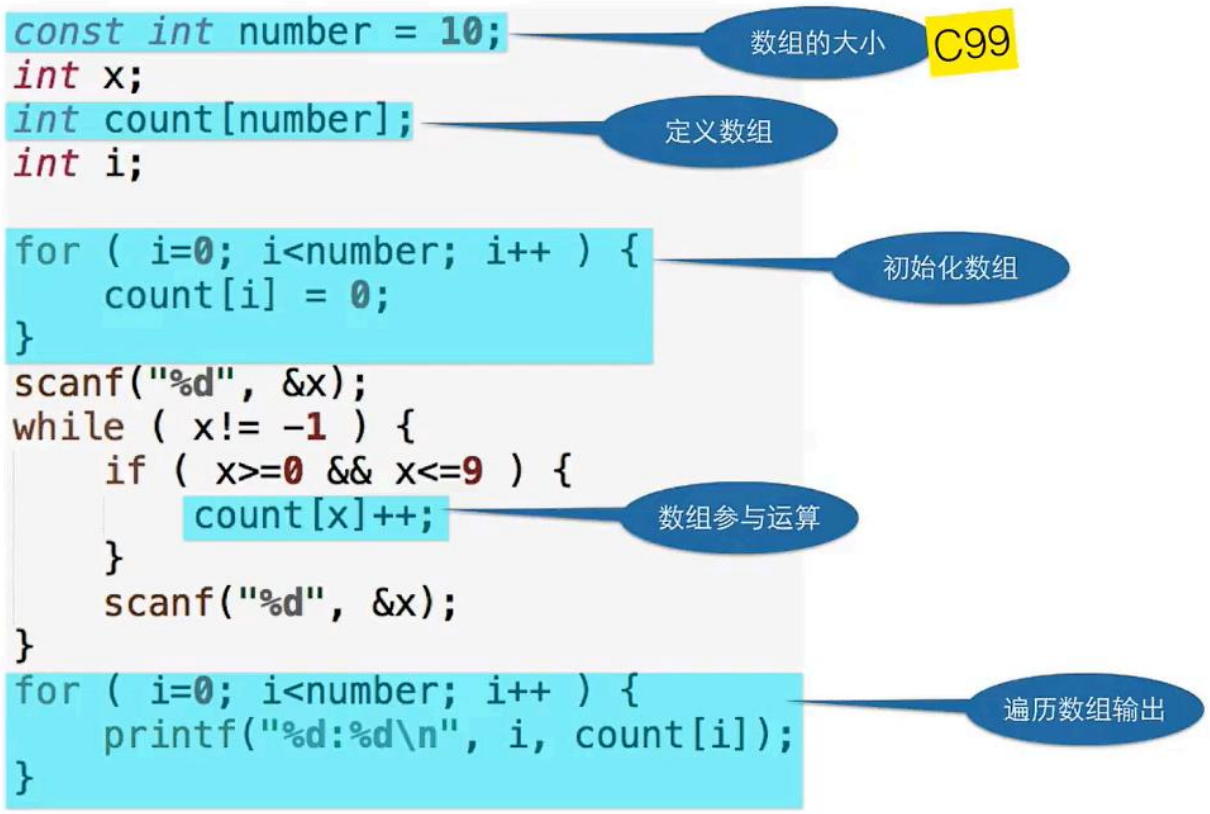
55数组的例子_统计个数

- 写一个程序，输入数量不确定的`[0,9]`范围内的整数，统计每一种数字出现的次数，输入-1表示结束

```c
#include <stdio.h>

int main(void)
{
    const int number = 10;
    int x;
    int count[number];
    int i;

    for ( i=0; i<number; i++ ) {
        count[i] = 0;
    }
    scanf("%d", &x);
    while ( x!= -1 ) {
        if ( x>=0 && x<=9 ) {
            count[x] ++;
        }
        scanf("%d", &x);
    }
    for ( i=0; i<10; i++ ) {
        printf("%d:%d\n", i, count[i]);
    }
    return 0;
}
```




```c
#define _CRT_SECURE_NO_WARNINGS 1
#include <stdio.h>

int main(void)
{
    //const int number = 10;
    int x;
    //int count[number];
    int count[10];//在VS的C语言标准里，定义数组的长度必须是一个明确的数字（比如`10`），不能用 `const int` 声明的变量当作数组长度。

    int i;

    for (i = 0;i < 10;i++) {
        count[i] = 0;
    }
    scanf("%d", &x);
    while (x != -1) {
        if (x >= 0 && x <= 9) {
            count[x]++;
        }
        scanf("%d", &x);
    }
    for (i = 0;i < 10;i++) {
        printf("%d:%d\n", i, count[i]);
    }
    return 0;
```

```
}
//输入1 5 1 2 1 -1
/*输出
0:0
1:3
2:1
3:0
4:0
5:1
6:0
7:0
8:0
9:0
*/
```

56数组运算

- 在一组给定的数据中，如何找出某个数据是否存在?

```c
#include <stdio.h>

/**
找出key在数组a中的位置
@param key 要寻找的数字
@param a 要寻找的数组
@param length 数组a的长度
@return 如果找到，返回其在a中的位置；如果找不到则返回-1
*/
int search(int key, int a[], int length);

int main(void)
{
    int a[] = {2,4,6,7,1,3,5,9,11,13,23,14,32};
    int x;
    int loc;
    printf("请输入一个数字: ");
    scanf("%d", &x);
    loc=search(x, a, sizeof(a)/sizeof(a[0]));
    if ( loc != -1 ) {
        printf("%d在第%d个位置上\n", x, loc);
    } else {
        printf("%d不存在\n", x);
    }

    return 0;
}

int search(int key, int a[], int length)
{
    int ret = -1;
    int i;
    for ( i=0; i< length; i++ ) {
        if ( a[i] == key ) {
            ret = i;
            break;
        }
    }
    return ret;
}
```

集成初始化时的定位



C99 ONLY!

```
int a[10] = {
    [0] = 2, [2] = 3, 6 ,
};
```

- 用[n]在初始化数据中给出定位
- 没有定位的数据接在前面的位置后面
- 其他位置的值补零
- 也可以不给出数组大小，让编译器算
- 特别适合初始数据稀疏的数组

数组的大小



- sizeof给出整个数组所占据的内容的大小，单位是字节

```
sizeof(a)/sizeof(a[0])
```

- sizeof(a[0])给出数组中单个元素的大小，于是相除就得到了数组的单元个数
- 这样的代码，一旦修改数组中初始的数据，不需要修改遍历的代码

数组的赋值



```
int a[] = {2,4,6,7,1,3,5,9,11,13,23,14,32};
int b[] = a;
```

- 数组变量本身不能被赋值
- 要把一个数组的所有元素交给另一个数组，必须采用遍历

```
for ( i=0; i<length; i++ ) {
    b[i] = a[i];
}
```

遍历数组

```
for ( i=0; i<length; i++ ) {
    b[i] = a[i];
}
```

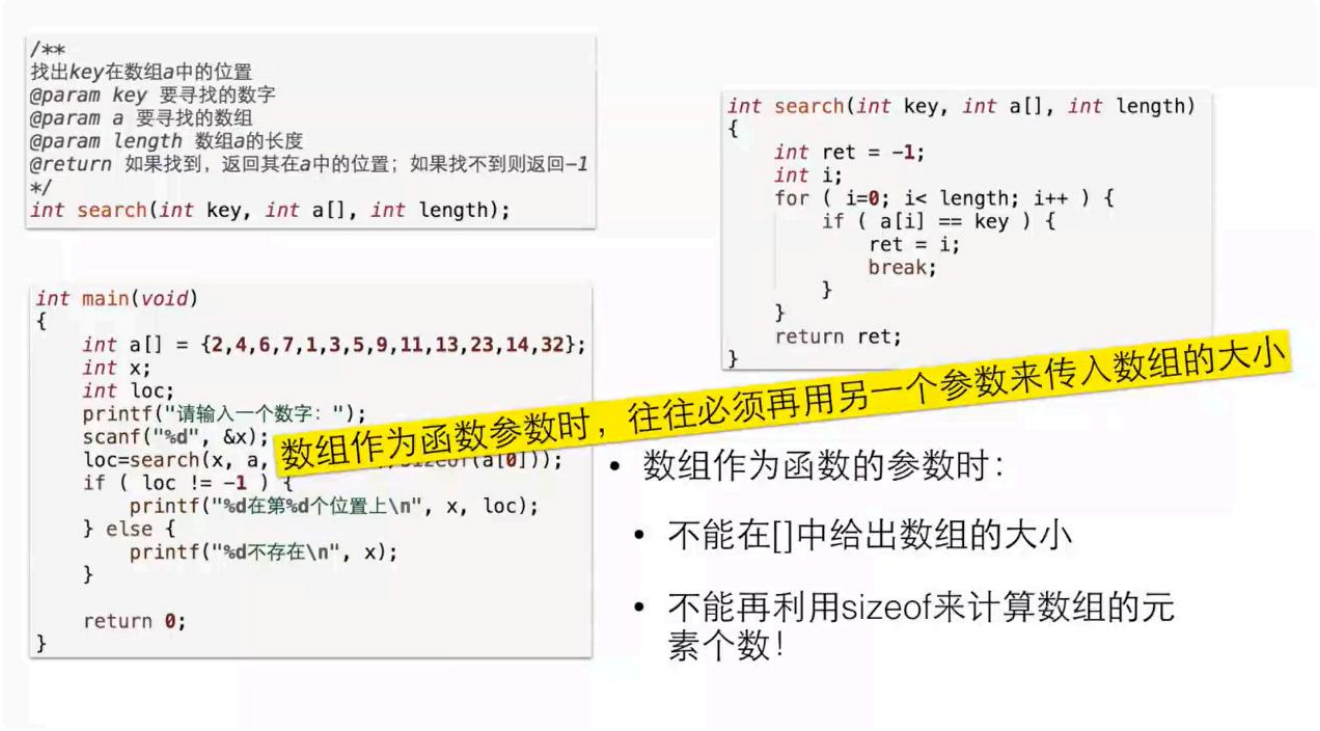
```
for ( i=0; i<number; i++ ) {
    count[i] = 0;
}
```

```
for ( i=0; i< length; i++ ) {
    if ( a[i] == key ) {
        ret = i;
        break;
    }
}
```

```
for ( i=0; i<cnt; i++ ) {
    if ( number[i] > average ) {
        printf("%d ", number[i]);
    }
}
```

```
for ( i=0; i<number; i++ ) {
    printf("%d:%d\n", i, count[i])
}
```

- 通常都是使用for循环，让循环变量i从0到<数组的长度，这样循环体内最大的i正好是数组最大的有效下标
- 常见错误是：
  - 循环结束条件是<=数组长度，或；
  - 离开循环后，继续用i的值来做数组元素的下标！



1. **数组传参规则**：把数组传给函数时，必须额外传一个整数参数（比如 `length`），用来告诉函数数组有多长。
2. **不能省略长度**：函数定义里的 `int a[]` 中括号里不能写具体数字（比如 `a[10]` 是错的），写啥编译器都会忽略，不如老老实实加个 `length` 参数。
3. **禁区**：在函数内部，千万不要用 `sizeof` 去算传进来的数组长度，算不准！

57数组例子_素数

判断素数

```c
int isPrime(int x);

int main(void)
{
    int x;
    scanf("%d", &x);
    if ( isPrime(x) ) {
        printf("%d是素数\n", x);
    } else {
        printf("%d不是素数\n", x);
    }
    return 0;
}
```

从2到x-1测试是否可以整除

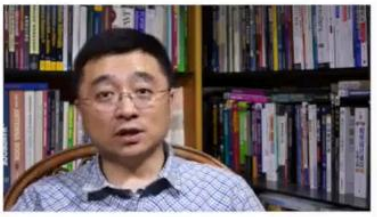
```c
int isPrime(int x)
{
    int ret = 1;
    int i;
    if ( x == 1 ) ret = 0;
    for ( i=2; i<x; i++ ) {
        if ( x % i == 0 ) {
            ret = 0;
            break;
        }
    }
    return ret;
}
```



- 对于n要循环n-1遍
- 当n很大时就是n遍

去掉偶数后，从3到x-1，每次加2

```
int isPrime(int x)
{
    int ret = 1;
    int i;
    if ( x == 1 ||
         (x%2 == 0 && x!=2) )
        ret = 0;
    for ( i=3; i<x; i+=2 ) {
        if ( x % i == 0 ) {
            ret = 0;
            break;
        }
    }
    return ret;
}
```



- 如果x是偶数，立刻
- 否则要循环(n-3)/2+1遍
  - 当n很大时就是n/2遍

无须到x-1，到sqrt(x)就够了

```
int isPrime(int x)
{
    int ret = 1;
    int i;
    if ( x == 1 ||
         (x%2 == 0 && x!=2) )
        ret = 0;
    for ( i=3; i<sqrt(x); i+=2 ) {
        if ( x % i == 0 ) {
            ret = 0;
            break;
        }
    }
    return ret;
}
```



- 只需要循环sqrt(x)遍

判断是否能被已知的且小的素数整除

```c
int main(void)
{
    const int number = 100;
    int prime[number] = {2};
    int count = 1;
    int i = 3;
    while ( count < number ) {
        if ( isPrime(i, prime, count) ) {
            prime[count++] = i;
        }
        i++;
    }
    for ( i=0; i<number; i++ ) {
        printf("%d", prime[i]);
        if ( (i+1)%5 ) printf("\t");
        else printf("\n");
    }
    return 0;
}
```



```c
int isPrime(int x, int knownPrimes[], int numberOfKnownPrimes)
{
    int ret = 1;
    int i;
    for ( i=0; i<numberOfKnownPrimes; i++ ) {
        if ( x % knownPrimes[i] ==0 ) {
            ret = 0;
            break;
        }
    }
    return ret;
}
```

```c
#include <stdio.h>

int isPrime(int x, int knownPrimes[], int numberOfKnownPrimes);

int main(void)
{
    const int number = 10;
    int prime[number] = {2};
    int count = 1;
    int i = 3;

    while ( count < number ) {
        if ( isPrime(i, prime, count) ) {
            prime[count++] = i;
        }
        {
            printf("i=%d \tcnt=%d\t",i, count);
            int i;
            for ( i=0; i<number; i++ ) {
                printf("%d\t", prime[i]);
            }
            printf("\n");
        }
        i++;
    }
    for ( i=0; i<number; i++ ) {
        printf("%d", prime[i]);
        if ( (i+1)%5 ) printf("\t");
        else printf("\n");
    }
    return 0;
}
```

构造素数表

- 欲构造n以内的素数表

1. 令x为2
2. 将2x、3x、4x直至ax<n的数标记为非素数
3. 令x为下一个没有被标记为非素数的数，重复2；直到所有的数都已经尝试完毕

进一步：

构造素数表

- 欲构造n以内（不含）的素数表

1. 开辟prime[n]，初始化其所有元素为1，prime[x]为1表示x是素数
2. 令x=2
3. 如果x是素数，则对于(i=2;x*i<n;i++)令prime[i*x]=0
4. 令x++，如果x<n，重复3，否则结束

```c
#include <stdio.h>

int main()
{
    const int maxNumber = 25;
    int isPrime[maxNumber];
    int i;
    int x;
    for ( i=0; i<maxNumber; i++ ) {
        isPrime[i] = 1;
    }
    for ( x=2; x<maxNumber; x++ ) {
        if ( isPrime[x] ) {
            for ( i=2; i*x<maxNumber; i++ ) {
                isPrime[i*x] = 0;
            }
        }
    }
    for ( i=2; i<maxNumber; i++ ) {
        if ( isPrime[i] ) {
            printf("%d\t", i);
        }
    }
    printf("\n");

    return 0;
}
```

```c
            for ( i=2; i*x<maxNumber; i++ ) {
                isPrime[i*x] = 0;
            }
        }
        //  for test
```

```
2   1   1   0   1   0   1   0   1   0   1   0   1   0   1   0   1   0   1   0   1   0   1   0
3   1   1   0   1   0   1   0   0   0   1   0   1   0   0   0   1   0   1   0   0   0   1   0
4   1   1   0   1   0   1   0   0   0   1   0   1   0   0   0   1   0   1   0   0   0   1   0
5   1   1   0   1   0   1   0   0   0   1   0   1   0   0   0   1   0   1   0   0   0   1   0
6   1   1   0   1   0   1   0   0   0   1   0   1   0   0   0   1   0   1   0   0   0   1   0
7   1   1   0   1   0   1   0   0   0   1   0   1   0   0   0   1   0   1   0   0   0   1   0
8   1   1   0   1   0   1   0   0   0   1   0   1   0   0   0   1   0   1   0   0   0   1   0
9   1   1   0   1   0   1   0   0   0   1   0   1   0   0   0   1   0   1   0   0   0   1   0
10  1   1   0   1   0   1   0   0   0   1   0   1   0   0   0   1   0   1   0   0   0   1   0
11  1   1   0   1   0   1   0   0   0   1   0   1   0   0   0   1   0   1   0   0   0   1   0
12  1   1   0   1   0   1   0   0   0   1   0   1   0   0   0   1   0   1   0   0   0   1   0
13  1   1   0   1   0   1   0   0   0   1   0   1   0   0   0   1   0   1   0   0   0   1   0
14  1   1   0   1   0   1   0   0   0   1   0   1   0   0   0   1   0   1   0   0   0   1   0
15  1   1   0   1   0   1   0   0   0   1   0   1   0   0   0   1   0   1   0   0   0   1   0
16  1   1   0   1   0   1   0   0   0   1   0   1   0   0   0   1   0   1   0   0   0   1   0
17  1   1   0   1   0   1   0   0   0   1   0   1   0   0   0   1   0   1   0   0   0   1   0
18  1   1   0   1   0   1   0   0   0   1   0   1   0   0   0   1   0   1   0   0   0   1   0
19  1   1   0   1   0   1   0   0   0   1   0   1   0   0   0   1   0   1   0   0   0   1   0
20  1   1   0   1   0   1   0   0   0   1   0   1   0   0   0   1   0   1   0   0   0   1   0
21  1   1   0   1   0   1   0   0   0   1   0   1   0   0   0   1   0   1   0   0   0   1   0
22  1   1   0   1   0   1   0   0   0   1   0   1   0   0   0   1   0   1   0   0   0   1   0
23  1   1   0   1   0   1   0   0   0   1   0   1   0   0   0   1   0   1   0   0   0   1   0
24  1   1   0   1   0   1   0   0   0   1   0   1   0   0   0   1   0   1   0   0   0   1   0
2   3   5   7   11  13  17  19  23
[Finished in 0.2s]
```

很多行同时输出

构造素数表

```
const int maxNumber = 25;
int isPrime[maxNumber];
int i;
int x;
for ( i=0; i<maxNumber; i++ ) {
    isPrime[i] = 1;
}
for ( x=2; x<maxNumber; x++ ) {
    if ( isPrime[x] ) {
        for ( i=2; i*x<maxNumber; i++ ) {
            isPrime[i*x] = 0;
        }
    }
}
for ( i=2; i<maxNumber; i++ ) {
    if ( isPrime[i] ) {
        printf("%d\t", i);
    }
}
printf("\n");
```

- 算法不一定和人的思考方式相同

58二维数组

二维数组

- int a[3][5];
- 通常理解为a是一个3行5列的矩阵

a[0][0]	a[0][1]	a[0][2]	a[0][3]	a[0][4]
a[1][0]	a[1][1]	a[1][2]	a[1][3]	a[1][4]
a[2][0]	a[2][1]	a[2][2]	a[2][3]	a[2][4]

二维数组的遍历



```
for ( i=0; i<3; i++ ) {
    for ( j=0; j<5; j++ ) {
        a[i][j] = i*j;
    }
}
```

- a[i][j]是一个int
- 表示第i行第j列上的单元
  - a[j,j]是什么?

二维数组的初始化



```
int a[][5] = {
    {0,1,2,3,4},
    {2,3,4,5,6},
};
```

- 列数是必须给出的，行数可以由编译器来数
- 每行一个{}，逗号分隔
- 最后的逗号可以存在，有古老的传统
- 如果省略，表示补零
- 也可以用定位（ * C99 ONLY）

tic-tac-toe游戏



- 读入一个3X3的矩阵，矩阵中的数字为1表示该位置上有一个X，为0表示为O
- 程序判断这个矩阵中是否有获胜的一方，输出表示获胜一方的字符X或O，或输出无人获胜



本页图片来源：wikipedia.org

读入矩阵



```
const int size = 3;
int board[size][size];
int i,j;
int numOfX;
int numOfO;
int result = -1;     //  -1:没人赢，1:X赢，0:O赢

//  读入矩阵
for ( i=0; i<size; i++ ) {
    for ( j=0; j<size; j++ ) {
        scanf("%d", &board[i][j]);
    }
}
```

检查行

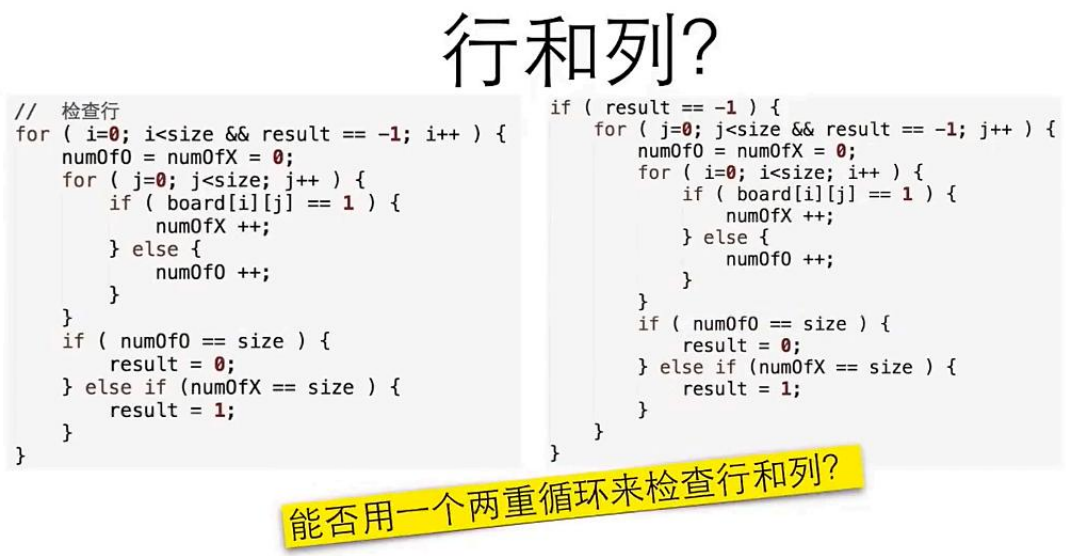
```
//  检查行
for ( i=0; i<size && result == -1; i++ ) {
    numOf0 = numOfX = 0;
    for ( j=0; j<size; j++ ) {
        if ( board[i][j] == 1 ) {
            numOfX ++;
        } else {
            numOf0 ++;
        }
    }
    if ( numOf0 == size ) {
        result = 0;
    } else if (numOfX == size ) {
        result = 1;
    }
}
```



检查列

```
if ( result == -1 ) {
    for ( j=0; j<size && result == -1; j++ ) {
        numOf0 = numOfX = 0;
        for ( i=0; i<size; i++ ) {
            if ( board[i][j] == 1 ) {
                numOfX ++;
            } else {
                numOf0 ++;
            }
        }
        if ( numOf0 == size ) {
            result = 0;
        } else if (numOfX == size ) {
            result = 1;
        }
    }
}
```





合并后的代码长这样（伪代码）：

c 复制 下载

```c
for ( i = 0; i < size && result == -1; i++ ) {
    int row0 = 0, rowX = 0; // 记录第i行的0和X数量
    int col0 = 0, colX = 0; // 记录第i列的0和X数量

    for ( j = 0; j < size; j++ ) {
        // 检查第 i 行
        if ( board[i][j] == 1 ) rowX++; else row0++;
        // 检查第 i 列
        if ( board[j][i] == 1 ) colX++; else col0++;
    }

    // 这一行/列查完了，看看有没有人赢
    if ( row0 == size || col0 == size ) result = 0;
    if ( rowX == size || colX == size ) result = 1;
}
```

59取地址运算_&运算符取得变量的地址

sizeof
- 是一个运算符，给出某个类型或变量在内存中所占据的字节数
- sizeof(int)、sizeof(i)

运算符&
- scanf("%d",&i);里的&
- 获得变量的地址，它的操作数必须是变量
- int i; printf("%x",&i);
- 地址的大小是否与int相同取决于编译器
- int i; printf("%p",&i);

&不能取的地址
• &不能对没有地址的东西取地址
• &(a+b)?
• &(a++)?
• &(++a)?

试试这些&
• 变量的地址
• 相邻的变量的地址
• &的结果的sizeof
• 数组的地址
• 数组单元的地址
• 相邻的数组单元的地址

60指针_指针变量就是记录地址的变量

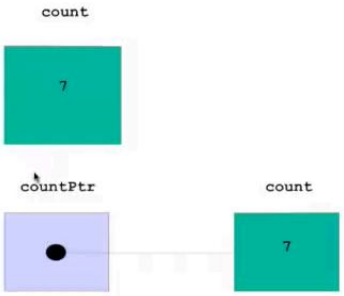
scanf
• 如果能够将取得的变量的地址传递给一个函数，能否通过这个地址在那个函数内访问这个变量?
• scanf("%d",&i);
• scanf()的原型应该是怎样的?我们需要一个参数能保存别的变量的地址，如何表达能够保存地址的变量?

指针
• 就是保存地址的变量
int i;
int *p = &i;*
int p,q;
int *p,q;

指针变量



- 变量的值是内存的地址
  - 普通变量的值是实际的值
  - 指针变量的值是具有实际值的变量的地址



作为参数的指针

- void f(int * p);
- 在被调用的时候得到了某个变量的地址:
- int i=0;f(&i);
- 在函数里面可以通过这个指针访问外面的这个i

8.2.c

```c
#include <stdio.h>

void f(int *p);

int main(void)
{
    int i = 6;
    printf("&i=%p\n", &i);
    f(&i);

    return 0;
}

void f(int *p)
{
    printf(" p=%p\n", p);
}
```

```
&i=0xbff17d70
 p=0xbff17d70
[Finished in 0.7s]
```

访问那个地址上的变量

• `是一个单目运算符，用来访问指针的值所表示的地址上的变量 • 可以做右值也可以做左值 • intk= *p; • `p=k+1;

传入地址

• 为什么

• int i;

scanf("%d", i);

• 编译没有报错?

✅ 正确写法

c 复制 下载

```c
int i;
scanf("%d", &i); // 拿到 i 的门牌号，精准写入
```

🔍 为什么编译不报错，但运行会崩溃？（底层原理）

1. **编译器的“偷懒”**：在 C 语言底层，地址和整数长得一样（都是一串十六进制数字）。`scanf` 以为你传进去的那个乱七八糟的垃圾值（比如 `i` 未初始化的默认值）是个合法的地址，所以放行了。
2. **运行时的“暴毙”**：程序一跑，拿着这串假地址去内存里乱写，操作系统立马发现非法入侵，直接给你扔出一个 **段错误（Segmentation fault）**，程序当场闪退。

61指针的使用_指针有什么用

指针应用场景一

- 交换两个变量的值

```
void swap(int *pa, int *pb)
{
    int t = *pa;
    *pa = *pb;
    *pb = t;
}
```

```c
#include <stdio.h>

void swap(int *pa, int *pb);

int main(void)
{
    int a = 5;
    int b = 6;
    swap(&a, &b);
    printf("a=%d,b=%d\n", a,b);

    return 0;
}

void swap(int *pa, int *pb)
{
    int t = *pa;
    *pa = *pb;
    *pb = t;
}
```

```
a=6,b=5
[Finished in 0.2s]
```

指针应用场景二

- 函数返回多个值，某些值就只能通过指针返回
  - 传入的参数实际上是需要保存带回的结果的变量

minmax.c

```c
void minmax(int a[], int len, int *max, int *min);

int main(void)
{
    int a[] = {1,2,3,4,5,6,7,8,9,12,13,14,16,17,21,23,55,};
    int min,max;
    minmax(a, sizeof(a)/sizeof(a[0]), &min, &max);
    printf("min=%d,max=%d\n", min, max);

    return 0;
}

void minmax(int a[], int len, int *min, int *max)
{
    int i;
    *min = *max=a[0];
    for ( i=1; i<len; i++ ) {
        if ( a[i] < *min ) {
            *min = a[i];
        }
        if ( a[i] > *max ) {
            *max = a[i];
        }
    }
}
```

```
min=1,max=55
[Finished in 0.2s]
```

指针应用场景二b



- 函数返回运算的状态，结果通过指针返回
- 常用的套路是让函数返回特殊的不属于有效范围内的值来表示出错：
  - -1或0（在文件操作会看到大量的例子）
- 但是当任何数值都是有效的可能结果时，就得分开返回了
  - 后续的语言（C++,Java）采用了异常机制来解决这个问题

divide.c

```c
#include <stdio.h>

/**
    @return 如果除法成功，返回1；否则返回0
*/
int divide(int a, int b, int *result);

int main(void)
{
    int a=5;
    int b=2;
    int c;
    if ( divide(a,b,&c) ) {
        printf("%d/%d=%d\n", a, b, c);
    }
    return 0;
}

int divide(int a, int b, int *result)
{
    int ret = 1;
    if ( b == 0 ) ret = 0;
    else {
        *result = a/b;
    }
    return ret;
}

```

指针最常见的错误:

• 定义了指针变量，还没有指向任何变量，就开始使用指针

62指针与数组_为什么数组传进函数后的sizeof不对了

传入函数的数组成了什么?

```
int isPrime(int x, int knownPrimes[], int numberOfKnownPrimes)
{
    int ret = 1;
    int i;
    for ( i=0; i<numberOfKnownPrimes; i++ ) {
        if ( x % knownPrimes[i] ==0 ) {
            ret = 0;
            break;
        }
    }
    return ret;
}
```

- 函数参数表中的数组实际上是指针
  - sizeof(a) == sizeof(int*)
  - 但是可以用数组的运算符[]进行运算

数组参数

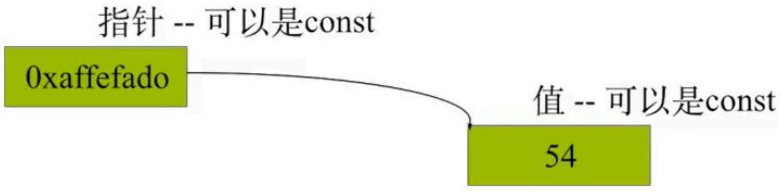
- 以下四种函数原型是等价的:
  - int sum(int *ar, int n);
  - int sum(int *, int);
  - int sum(int ar[], int n);
  - int sum(int [], int);

数组变量是特殊的指针

- 数组变量本身表达地址，所以
  - int a[10]; int*p=a; // 无需用&取地址
  - 但是数组的单元表达的是变量，需要用&取地址
  - a == &a[0]
- []运算符可以对数组做，也可以对指针做：
  - p[0] <==> a[0]
- *运算符可以对指针做，也可以对数组做：
  - *a = 25;
- 数组变量是const的指针，所以不能被赋值
  - int a[] <==> int * const a=....

63指针与const_指针本身和所指的变量都可能const

指针与const



本视频只适用于C99

1.指针是const

- 表示一旦得到了某个变量的地址，不能再指向其他变量
- int * const q=&i;//q是const
- * q=26;//OK
- q++; //ERROR

2.所指是const

• 表示不能通过这个指针去修改那个变量(并不能使得那个变量成为const)
• const int *p = &i;
• *p=26;//ERROR!(*p)是const
• i=26;//OK
• p=&j;//OK

3.这些是啥意思?

int i;
const int *p1=&i ;*
int const p2=&i ;
int *cohst p3 = &i ;
判断哪个被const了的标志是const在*的前面还是后面

4.转换

• 总是可以把一个非const的值转换成const的

```
void f(const int* x);
int a = 15;
f(&a); // ok
const int b = a ;

f(&b); //ok
b = a + 1; // Error!
```

• 当要传递的参数的类型比地址大的时候，这是常用的手段：既能用比较少的字节数传递值给参数，又能避免函数对外面的变量的修改

5.const数组

• const int a[] = {1,2,3,4,5,6,};
• 数组变量已经是const的指针了，这里的const表明数组的每个单元都是const int
• 所以必须通过初始化进行赋值

6.保护数组值

• 因为把数组传入函数时传递的是地址，所以那个函数内部可以修改数组的值
• 为了保护数组不被函数破坏，可以设置参数const
• int sum(const int a[], int length);

64指针运算

1+1=2?

```c
#include <stdio.h>

int main(void)
{
    char ac[] = {0,1,2,3,4,5,6,7,8,9,};
    char *p = ac;
    printf("p  =%p\n", p);
    printf("p+1=%p\n", p+1);
    printf("*(p+1)=%d\n", *(p+1));
    *(p+n) <-> ac[n]

    int ai[] = {0,1,2,3,4,5,6,7,8,9,};
    int *q = ai;
    printf("q  =%p\n", q);
    printf("q+1=%p\n", q+1);
    printf("*(q+1)=%d\n", *(q+1));
    return 0;
}
```

```
*(p+1)=1
q  =0xbffc4d2c
q+1=0xbffc4d30
*(q+1)=1
[Finished in 0.1s]
```

指针计算

- 这些算术运算可以对指针做:
- 给指针加、减一个整数(+,+=,-,=)
- 递增递减(++/)
- 两个指针相减

```c
#include <stdio.h>

int main(void)
{
    char ac[] = {0,1,2,3,4,5,6,7,8,9,};
    char *p = &ac[0];
    char *p1 = &ac[5];
    printf("p  =%p\n", p);
    printf("p+1=%p\n", p+1);
    printf("p1-p=%d\n", p1-p);
    int ai[] = {0,1,2,3,4,5,6,7,8,9,};
    int *q = ai;
    int *q1 = &ai[6];
    printf("q  =%p\n", q);
    printf("q1 =%p\n", q1);
    printf("q1-q=%d\n", q1-q);
    return 0;
}
```

```
p1-p=5
q  =0xbff11d28
q1 =0xbff11d40
q1-q=6
[Finished in 0.1s]
```

`*`p++

- 取出p所指的那个数据来，完事之后顺便把p移到下一个位置去
- `*`的优先级虽然高，但是没有++高
- 常用于数组类的连续空间操作
- 在某些CPU上，这可以直接被翻译成一条汇编指令

指针比较

- <,<=, == ,>,>=,!=都可以对指针做
- 比较它们在内存中的地址
- 数组中的单元的地址肯定是线性递增的

0地址

- 当然你的内存中有0地址，但是0地址通常是个不能随便碰的地址
- 所以你的指针不应该具有0值
- 因此可以用0地址来表示特殊的事情:
- 返回的指针是无效的
- 指针没有被真正初始化(先初始化为0)
- NULL是一个预定定义的符号，表示0地址
- 有的编译器不愿意你用0来表示0地址

指针的类型

- 无论指向什么类型，所有的指针的大小都是一样的，因为都是地址
- 但是指向不同类型的指针是不能直接互相赋值的
- 这是为了避免用错指针

```c
#include <stdio.h>

int main(void)
{
    char ac[] = {0,1,2,3,4,5,6,7,8,9,-1};
    char *p = &ac[0];
    int i;
    for ( i=0; i<sizeof(ac)/sizeof(ac[0]); i++ ) {
        printf("%d\n", ac[i]);
    }
    // for ( p=ac; *p!=-1 ; ) {
    while ( *p != -1 ) {
        printf("%d\n", *p++);
    }
    int ai[] = {0,1,2,3,4,5,6,7,8,9,};
    int *q = ai;
    q = p;
    return 0;
}
```

```
/Users/wengkai/cc/9.5.c:17:4: warning: incompatible pointer types assigning to 'int *' from 'char *' [-Wincompatib
        q = p;
          ^ ~
1 warning generated.
[Finished in 0.1s]
```

指针的类型转换

- void * 表示不知道指向什么东西的指针
- 计算时与char * 相同(但不相通)
- 指针也可以转换类型
- int * p = &i; void * q = (void *)p;
- 这并没有改变p所指的变量的类型，而是让后人用不同的眼光通过p看它所指的变量
- 我不再当你是int啦，我认为你就是个void!

用指针来做什么

- 需要传入较大的数据时用作参数
- 传入数组后对数组做操作
- 函数返回不止一个结果:
- 需要用函数来修改不止一个变量

65动态内存分配

输入数据

- 如果输入数据时，先告诉你个数，然后再输入，要记录每个数据
- C99可以用变量做数组定义的大小，C99之前呢?
- int `*`a = (int*)malloc(n`sizeof(int));

```
MALLOC(3)                BSD Library Functions Manual

NAME
     calloc, free, malloc, realloc, reallocf, valloc -- memory allocation

SYNOPSIS
     #include <stdlib.h>

     void *
     calloc(size_t count, size_t size);

     void
     free(void *ptr);

     void *
     malloc(size_t size);

     void *
     realloc(void *ptr, size_t size);

     void *
:
```

```c
#include <stdio.h>
#include <stdlib.h>

int main(void)
{
    int number;
    int* a;
    int i;
    printf("输入数量: ");
    scanf("%d", &number);
    // int a[number];
    a = (int*)malloc(number*sizeof(int));
    for (i=0; i<number; i++ ) {
        scanf("%d", &a[i]);
    }
    for ( i=number-1; i>=0; i-- ) {
        printf("%d ", a[i]);
    }
    free(a);

    return 0;
}
```

malloc

`#include` <stdlib.h>

void *malloc(size_t size);*

- *向malloc申请的空间的大小是以字节为单位的*
- *返回的结果是void*，需要类型转换为自己需要的类型
- (int*)malloc(n*sizeof(int))

没空间了?

- 如果申请失败则返回0，或者叫做NULL
- 你的系统能给你多大的空间?

```c
#include <stdio.h>
#include <stdlib.h>

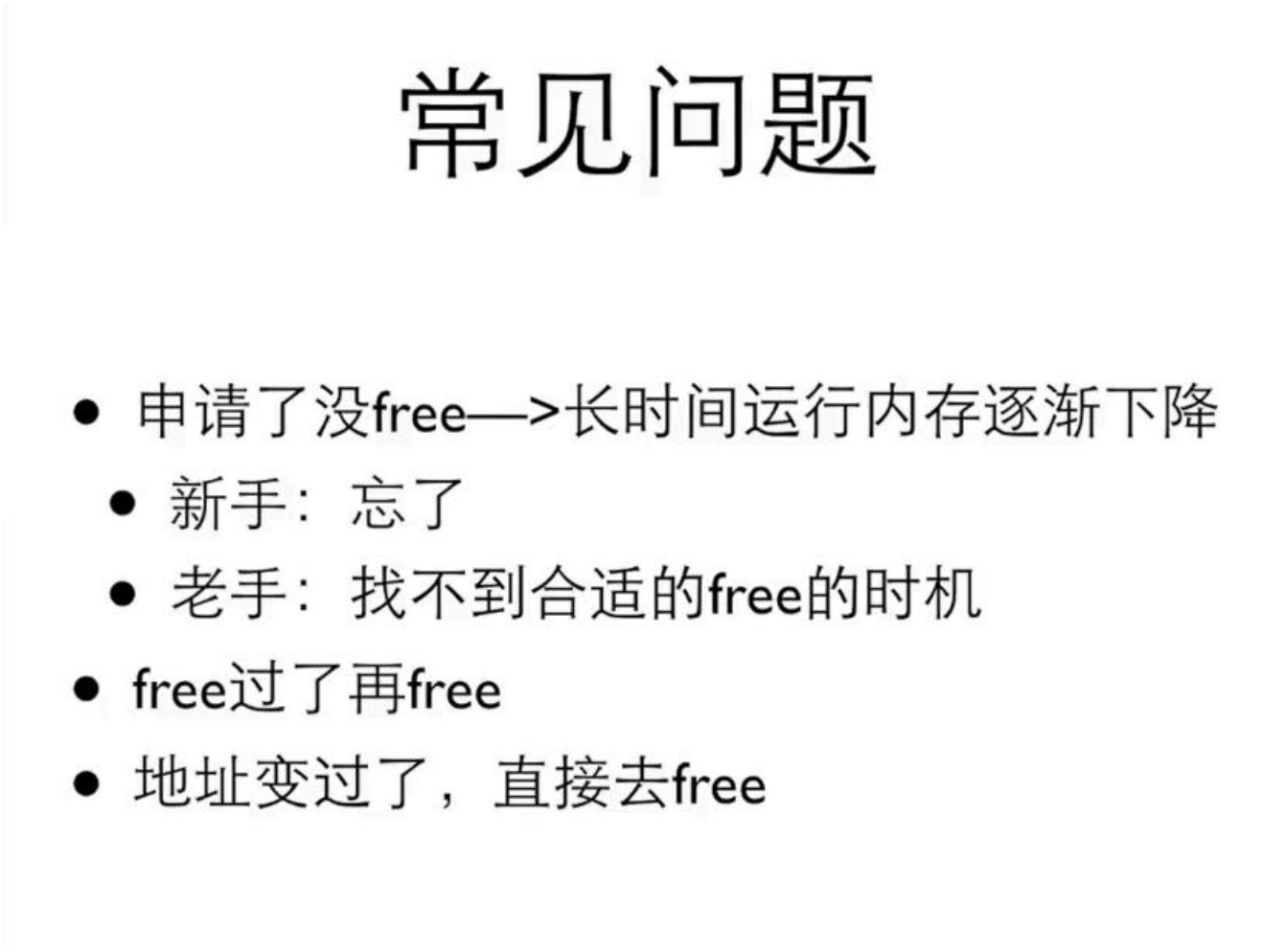
int main(void)
{
    void *p;
    int cnt = 0;
    while ( (p=malloc(100*1024*1024)) ) {
        cnt++;
    }
    printf("分配了%d00MB的空间\n", cnt);

    return 0;
}
```

```
.7(20083,0xa039f1a8) malloc: *** mach_vm_map(size=104857600) failed (error code=3)
*** error: can't allocate region
*** set a breakpoint in malloc_error_break to debug
分配了3300MB的空间
[Finished in 0.1s]
```

free()

- 把申请得来的空间还给“系统”
- 申请过的空间，最终都应该要还
- 混出来的，迟早都是要还的
- 只能还申请来的空间的首地址



常见问题
• 申请了没free—>长时间运行内存逐渐下降
• 新手:忘了
• 老手:找不到合适的free的时机
• free过了再free
• 地址变过了，直接去free

66字符串

1.字符数组
char word[]= {'H','e','l','l','o','!');

word[0]	**H**
word[1]	e
word[2]	l
word[3]	l
word[4]	o
word[5]	!

这不是C语言的字符串，因为不能用字符串的方式做计算

2.字符串

char word[]= ("H','e','l','l','o','!,'\0');

word[0]	H
word[1]	e
word[2]	l
word[3]	l
word[4]	o
word[5]	!
word[6]	\0

- 以0(整数0)结尾的一串字符
- 0或'\0'是一样的，但是和'0'不同
- 0标志字符串的结束，但它不是字符串的一部分
- 计算字符串长度的时候不包含这个0
- 字符串以数组的形式存在，以数组或指针的形式访问
- 更多的是以指针的形式
- string.h里有很多处理字符串的函数

3.字符串变量

- char * str = "Hello" ;
- char word[] = "Hello" ;
- char line[10] = "Hello" ;

4字符串常量:

- "Hello"
- "Hello"会被编译器变成一个字符数组放在某处，这个数组的长度是6，结尾还有表示结束的0
- 两个相邻的字符串常量会被自动连接起来

5.字符串:

- C语言的字符串是以字符数组的形态存在的
- 不能用运算符对字符串做运算
- 通过数组的方式可以遍历字符串
- 唯一特殊的地方是字符串字面量可以用来初始化字符数组
- 以及标准库提供了一系列字符串函数

67字符串变量

1.字符串常量

char *s = "Hello, world!" ;*

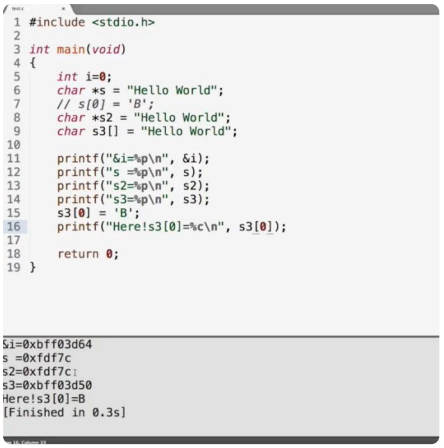
• s是一个指针，初始化为指向一个字符串常量

*• 由于这个常量所在的地方，所以实际上s是const char`*s，但是由于历史的原因，编译器接受不带const的写法

• 但是试图对s所指的字符串做写入会导致严重的后果:

• 如果需要修改字符串，应该用数组:

char s[] = "Hello, world!" ;



```
#include <stdio.h>

int main(void)
{
    int i=0;
    char *s = "Hello World";
    // s[0] = 'B';
    char *s2 = "Hello World";
    char s3[] = "Hello World";

    printf("&i=%p\n", &i);
    printf("s =%p\n", s);
    printf("s2=%p\n", s2);
    printf("s3=%p\n", s3);
    s3[0] = 'B';
    printf("Here!s3[0]=%c\n", s3[0]);

    return 0;
}
```

```
&i=0xbff03d64
s =0xfdf7c
s2=0xfdf7c
s3=0xbff03d50
Here!s3[0]=B
[Finished in 0.3s]
```

2.指针还是数组?

• char `*` str = "Hello" ;

• char word[]= "Hello" ;

• 数组:这个字符串在这里

• 作为本地变量空间自动被回收

• 指针:这个字符串不知道在哪里

• 处理参数

• 动态分配空间

> • 如果要构造一个字符串—>数组
> • 如果要处理一个字符串—>指针

char `*` 是字符串?

• 字符串可以表达为char `*` 的形式

• char `*` 不一定是字符串

• 本意是指向字符的指针,可能指向的是字符的数组(就像int `*` 一样)

• 只有它所指的字符数组有结尾的0,才能说它所指的是字符串

68字符串输入输出

1.字符串赋值?

- char *t = "title" ;
- char *s ;
- s=t ;
- 并没有产生新的字符串,只是让指针s指向了t所指的字符串,对s的任何操作就是对t做的

2.字符串输入输出

- char string[8] ;
- scanf("%s", string) ;
- printf("%s",string) ;

- scanf读入一个单词(到空格、tab或回车为止)
- scanf是不安全的，因为不知道要读入的内容的长度

3.安全的输入

- char string[8];
- scanf("%7s", string);

- 在%和s之间的数字表示最多允许读入的字符的数量，这个数字应该比数组的大小小一
- 下一次scanf从哪里开始?

4.常见错误

- char *string ;
- scanf("%s", string) ;
- 以为char *是字符串类型，定义了一个字符串类型的变量string就可以直接使用了
- 由于没有对string初始化为0，所以不一定每次运行都出错

5.空字符串

- char buffer[100]="";
- 这是一个空的字符串,buffer[0] == '\0'
- char buffer[] =""; .
- 这个数组的长度只有1!

69字符串数组

1.字符串数组

- char **a
- a是一个指针，指向另一个指针，那个指针指向一个字符(串)
- char a[][]

```
printf("请输入月份: ");
int month;
scanf("%d", &month);
switch ( month ) {
      case 1: printf("January\n"); break;
      case 2: printf("February\n"); break;
      case 3: printf("March\n"); break;
      case 4: printf("April\n"); break;
      case 5: printf("May\n"); break;
      case 6: printf("June\n"); break;
      case 7: printf("July\n"); break;
      case 8: printf("August\n"); break;
      case 9: printf("September\n"); break;
      case 10:printf("October\n"); break;
      case 11:printf("November\n"); break;
      case 12:printf("December\n"); break;
}
```

今后可以用数组来做

2.程序参数

- int main(int argc, char const * argv[])
- argv[0]是命令本身
- 当使用Unix的符号链接时,反映符号链接的名字

70单字符输入输出

putchar

- int putchar(int c) ;
- 向标准输出写一个字符
- 返回写了几个字符,EOF(-1)表示写失败

getchar

- int getchar(void);
- 从标准输入读入一个字符
- 返回类型是int是为了返回EOF(-1)
- Windows->Ctrl-Z
- Unix一>Ctrl-D

71字符串函数strlen

string.h
• strlen
• strcmp
• strcpy
• strcat
• strchr
• strstr

strlen
• size_t strlen(const char * s) ;
• 返回s的字符串长度(不包括结尾的0)

72字符串函数strc

strcmp
• int strcmp(const char * s1, const char * s2) ;
• 比较两个字符串,返回:
• 0:s1 == s2
• 1:s1>s2
• -1:s1 < s2

73字符串函数strcpy

strcpy
• char *strcpy(char `restrict dst, const char *`restrict src) ;
• 把src的字符串拷贝到dst
• restrict表明src和dst不重叠(C99)
• 返回dst
• 为了能链起代码来

复制一个字符串:
• char * dst =(char*)malloc(strlen(src)+1) ;
strcpy(dst, src) ;

74字符串搜索函数

字符串中找字符
• char *strchr(const char `s, int c) ; • char * strrchr(const char *`s, int c) ;
• 返回NULL表示没有找到

- 如何寻找第2个?

```c
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

int main(int argc, char const *argv[])
{
    char s[] = "hello";
    char *p = strchr(s, 'l');
    char *t = (char*)malloc(strlen(p)+1);
    strcpy(t, p);
    printf("%s\n", t);
    free(t);

    return 0;
}
```

```
llo
[Finished in 0.2s]
```

这段代码非常经典，它完美融合了你最近刚学的**指针、字符串操作、以及动态内存分配**（`malloc`）。

简单来说，它的核心功能是：**在一个字符串中查找特定的字符，并将从这个字符开始到结尾的这一段"截取"出来，存入一个新的内存空间，最后打印出来。**

结合底部的输出 `llo`，我们逐行"解剖"一下它是怎么工作的：

逐行拆解：

1. **第 7 行**：`char s[] = "hello";`
   - 你在内存（栈区）里创建了一个字符数组 `s`，里面装着 `h e l l o \0`。
2. **第 8 行**：`char *p = strchr(s, 'l');`
   - `strchr` 是一个字符串查找函数。它会在 `s` 里面从左往右找**第一个**出现的字符 `'l'`。
   - 找到之后，返回指向这个 `'l'` 的**指针**，赋值给 `p`。
   - 此时，`p` 指向的是字符串的第二个 `l`（即 `s[2]` 的位置）。此时 `p` 背后跟着的字符串是 `"llo"`。
3. **第 9 行**：`char *t = (char*)malloc(strlen(p)+1);`
   - `strlen(p)`：计算从 `p` 开始到结尾的长度，也就是 `"llo"` 的长度，结果是 3。
   - `+1`：非常关键！因为 C 语言字符串必须以 `\0` 结尾，所以要多申请 1 个字节。
   - `malloc(...)`：向操作系统申请 4 个字节的**堆内存**。
   - `(char*)`：把申请到的无类型指针强转为字符指针，交给 `t` 保管。
4. **第 10 行**：`strcpy(t, p);`
   - 把 `p` 指向的字符串（`"llo"`）复制到 `t` 指向的这块新内存里。现在 `t` 里面变成了 `l l o \0`。
5. **第 11 行**：`printf("%s\n", t);`
   - 打印 `t` 指向的字符串。所以屏幕上输出了 `llo`。
6. **第 12 行**：`free(t);`
   - **极其关键的一步！** 用 `malloc` 申请的内存，系统不会自动回收。必须用 `free` 还回去，否则就会导致"内存泄漏"。这是考研和面试的绝对高频考点。

考研视角的"三大考点"：

1. **指针的跳跃能力**：`p` 并没有从头开始，而是通过 `strchr` 定位到了中间，这体现了指针"指哪打哪"的能力。以后学数据结构，链表节点的查找、跳过某个节点，全是这个逻辑。
2. **`strlen` 的计算陷阱**：很多人会忘记 `malloc` 时加 `1`。如果不加 1，复制过来的字符串没有 `\0`，打印时就会越界输出一堆乱码。
3. **内存管理（栈 vs 堆）**：`s[]` 是在栈上分配的（函数结束自动销毁）；`malloc` 的 `t` 是在堆上分配的（手动 `free` 才能销毁）。

字符串中找字符串

- char *strstr(const char `s1, const char `s2);*
- *char* strcasestr(const char `*` s1, const char `*` s2);

74结构类型

声明结构类型



```c
#include <stdio.h>

int main(int argc, char const *argv[])
{
    struct date {
        int month;
        int day;
        int year;
    };

    struct date today;

    today.month = 07;
    today.day = 31;
    today.year = 2014;

    printf("Today's date is %i-%i-%i.\n",
        today.year,today.month,today.day);

    return 0;
}
```

初学者最常见的
错误：漏了这个分号！

在函数内/外?

```c
#include <stdio.h>

struct date {
    int month;
    int day;
    int year;
};

int main(int argc, char const *argv[])
{
    struct date today;

    today.month = 07;
    today.day = 31;
    today.year = 2014;

    printf("Today's date is %i-%i-%i.\n",
        today.year,today.month,today.day);

    return 0;
}
```

- 和本地变量一样，在函数内部声明的结构类型只能在函数内部使用
- 所以通常在函数外部声明结构类型，这样就可以被多个函数所使用了

声明结构的形式



```
struct point {
    int x;
    int y;
};

struct point p1, p2;
```

p1 和 p2 都是point
里面有x和y的值

```
struct {
    int x;
    int y;
} p1, p2;
```

p1 和 p2都是一种无名结构，里面有x和y

```
struct  point {
    int x;
    int y;
} p1, p2;
```

p1和p2都是point
里面有x和y的值t

对于第一和第三种形式，都声明了结构point。但是第二种形式没有声明point，只是定义了两个变量

结构的初始化



```c
#include <stdio.h>

struct date {
    int month;
    int day;
    int year;
};

int main(int argc, char const *argv[])
{
    struct date today = {07,31,2014};
    struct date thismonth = {.month=7, .year=2014};

    printf("Today's date is %i-%i-%i.\n",
        today.year,today.month,today.day);
    printf("This month is %i-%i-%i.\n",
        thismonth.year,thismonth.month,thismonth.day);

    return 0;
}
```

结构成员



- 结构和数组有点像
- 数组用[]运算符和下标访问其成员
  - a[0] = 10;
- 结构用.运算符和名字访问其成员
  - today.day
  - student.firstName
  - p1.x
  - p1.y

结构运算



- 要访问整个结构，直接用结构变量的名字
- 对于整个结构，可以做赋值、取地址，也可以传递给函数参数
  - p1 = (struct point){5, 10}; // 相当于p1.x = 5; p1.y = 10;
  - p1 = p2; // 相当于p1.x = p2.x; p1.y = p2.y;

结构运算



- 要访问整个结构，直接用结构变量的名字
- 对于整个结构，可以做赋值、取地址，也可以传递给函数参数
  - p1 = (struct point){5, 10}; // 相当于p1.x = 5; p1.y = 10;
  - p1 = p2; // 相当于p1.x = p2.x; p1.y = p2.y;

数组无法做这两种运算！

```c
#include <stdio.h>

struct date {
    int month;
    int day;
    int year;
};

int main(int argc, char const *argv[])
{
    struct date today;

    today = (struct date){07,31,2014};

    struct date day;

    day = today;
    day.year = 2015;

    printf("Today's date is %i-%i-%i.\n",
        today.year,today.month,today.day);
    printf("The day's date is %i-%i-%i.\n",
        day.year,day.month,day.day);
```

```
Today's date is 2014-7-31.
The day's date is 2015-7-31.
[Finished in 0.3s]
```

结构指针



- 和数组不同，结构变量的名字并不是结构变量的地址，必须使用&运算符
- struct date *pDate = &today;

75结构与函数

结构作为函数参数



```
int numberOfDays(struct date d)
```

- 整个结构可以作为参数的值传入函数
- 这时候是在函数内新建一个结构变量，并复制调用者的结构的值
- 也可以返回一个结构
- 这与数组完全不同

输入结构

- 没有直接的方式可以一次scanf一个结构
- 如果我们打算写一个函数来读入结构
  - —>
- 但是读入的结构如何送回来呢？
- 记住C在函数调用时是传值的
  - 所以函数中的p与main中的y是不同的
  - 在函数读入了p的数值之后，没有任何东西回到main，所以y还是 {0, 0}

```
#include <stdio.h>

struct point  {
    int x;
    int y; };

void getStruct(struct point);
void output(struct point);
void main( )    {
    struct point y = {0, 0};
    getStruct(y);
    output(y); }

void getStruct(struct point p)   {
    scanf("%d", &p.x);
    scanf("%d", &p.y);
    printf("%d, %d", p.x, p.y); }

void output(struct point p)   {
    printf("%d, %d", p.x, p.y); }
```

解决的方案

- 之前的方案，把一个结构传入了函数，然后在函数中操作，但是没有返回回去
  - 问题在于传入函数的是外面那个结构的克隆体，而不是指针
    - 传入结构和传入数组是不同的
- 在这个输入函数中，完全可以创建一个临时的结构变量，然后把这个结构返回给调用者

```
void main( )
{
    struct point y = {0, 0};
    y = inputPoint( );
    output(y);
}
```

```
struct point inputPoint( )
{
    struct point temp;
    scanf("%d", &temp.x);
    scanf("%d", &temp.y);
    return temp;
}
```

指向结构的指针



```
struct date {
    int month;
    int day;
    int year;
} myday;

struct date *p = &myday;

(*p).month = 12;
p->month = 12;
```

- 用->表示指针所指的结构变量中的成员

结构指针参数



```
void main( )
{
   struct point y = {0, 0};
   inputPoint(&y);
   output(y);
}
```

```
struct point* inputPoint(struct point *p)
{
      scanf("%d", &(p->x));
      scanf("%d", &(p->y);
      return p;
}
```

76结构与函数

结构数组

```
struct date dates[100];
struct date dates[] = {
    {4,5,2005},{2,4,2005}};
```

结构中的结构

```
struct dateAndTime {
    struct date sdate;
    struct time stime;
};
```

嵌套的结构



```
struct point {
    int x;
    int y;
};
struct rectangle {
    struct point pt1;
    struct point pt2;
};
```

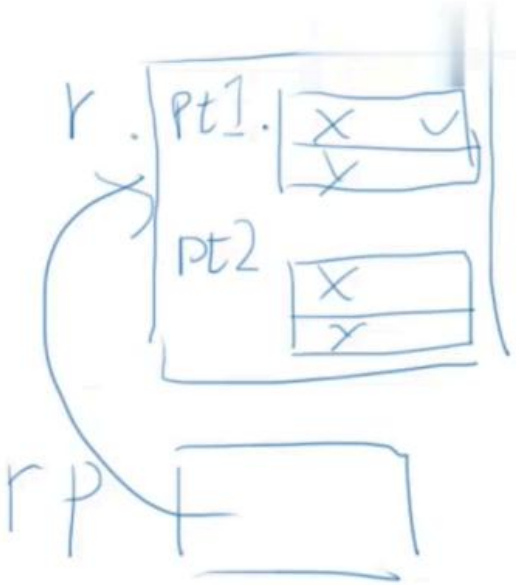
如果有变量
struct rectangle r;
就可以有：
r.pt1.x、r.pt1.y,
r.pt2.x 和 r.pt2.y

如果有变量定义：
struct rectangle r, *rp;
rp = &r;

那么下面的四种形式是等价的：
r.pt1.x
rp->pt1.x
(r.pt1).x
(rp->pt1).x
但是没有rp->pt1->x （因为pt1不是指针）

如果有变量定义：
struct rectangle r, *rp;
rp = &r;

那么下面的四种形式是等价的：
r.pt1.x
rp->pt1.x
(r.pt1).x
(rp->pt1).x
但是没有rp->pt1->x （因为pt1不是指针）



r.pt1.x
rp->pt1.x

结构中的结构的数组



```
#include <stdio.h>

struct point{
    int x;
    int y;
};

struct rectangle {
    struct point p1;
    struct point p2;
};

void printRect(struct rectangle r)
{
    printf("<%d, %d> to <%d, %d>\n", r.p1.x, r.p1.y, r.p2.x, r.p2.y);
}

int main(int argc, char const *argv[])
{
    int i;
    struct rectangle rects[ ] = {{{1, 2}, {3, 4}}, {{5, 6}, {7, 8}}}; // 2 rectangles
    for(i=0;i<2;i++) printRect(rects[i]);
}
```

77类型定义

Typedef



声明新的类型的名字

- 新的名字是某种类型的别名
- 改善了程序的可读性

```
typedef long int64_t;
typedef struct ADate {
  int month;
  int day;
  int year;
} Date;

int64_t i = 100000000000;
Date d = {9, 1, 2005};
```

重载已有的类型名字
新名字的含义更清晰
具有可移植性

简化了复杂的名字

typedef



```
typedef struct {
    int month;
    int day;
    int year;
} Date;
```

typedef



```
typedef  int Length; // Length就等价于int类型

typedef char* Strings[10];     // Strings 是10个字符串的数组的类型

typedef struct node {
   int data;
   struct node *next;
 } aNode;

或

typedef struct node aNode; // 这样用aNode 就可以代替 struct node
```

78联合

联合

选择:

成员是

- 一个 int i 还是
- 一个 char c

```
union AnElt {
   int   i;
   char  c;
} elt1, elt2;

elt1.i = 4;
elt2.c = 'a';
elt2.i = 0xDEADBEEF;
```

sizeof(union ...) =

sizeof(每个成员)的最大值



联合

- 存储
  - 所有的成员共享一个空间
  - 同一时间只有一个成员是有效的
  - union的大小是其最大的成员
- 初始化
  - 对第一个成员做初始化

79静态本地变量_能在函数结束后继续保有原值的本地变量

静态本地变量



- 在本地变量定义时加上static修饰符就成为静态本地变量
- 当函数离开的时候，静态本地变量会继续存在并保持其值
- 静态本地变量的初始化只会在第一次进入这个函数时做，以后进入函数时会保持上次离开时的值

静态本地变量



- 静态本地变量实际上是特殊的全局变量
- 它们位于相同的内存区域
- 静态本地变量具有全局的生存期，函数内的局部作用域
  - static在这里的意思是局部作用域（本地可访问）

80后记_返回指针的函数,使用全局变量的贴士

*返回指针的函数



- 返回本地变量的地址是危险的
- 返回全局变量或静态本地变量的地址是安全的
- 返回在函数内malloc的内存是安全的，但是容易造成问题
- 最好的做法是返回传入的指针

tips



- 不要使用全局变量来在函数间传递参数和结果
- 尽量避免使用全局变量
  - 丰田汽车的案子
- * 使用全局变量和静态本地变量的函数是线程不安全的

81宏定义

编译预处理指令



- #开头的是编译预处理指令
- 它们不是C语言的成分，但是C语言程序离不开它们
- #define用来定义一个宏

#define



- #define <名字> <值>
- 注意没有结尾的分号，因为不是C的语句
- 名字必须是一个单词，值可以是各种东西
- 在C语言的编译器开始编译之前，编译预处理程序（cpp）会把程序中的名字换成值
  - 完全的文本替换
- gcc —save-temps

宏



- 如果一个宏的值中有其他的宏的名字，也是会被替换的
- 如果一个宏的值超过一行，最后一行之前的行末需要加\
- 宏的值后面出现的注释不会被当作宏的值的一部分

没有值的宏



- #define _DEBUG
- 这类宏是用于条件编译的，后面有其他的编译预处理指令来检查这个宏是否已经被定义过了

预定义的宏



- __LINE__
- __FILE__
- __DATE__
- __TIME__
- __STDC__

82带参数的宏

像函数的宏



- #define cube(x) ((x)*(x)*(x))
- 宏可以带参数

错误定义的宏

- #define RADTODEG(x) (x * 57.29578)
- #define RADTODEG(x) (x) * 57.29578

带参数的宏的原则



- 一切都要括号
  - 整个值要括号
  - 参数出现的每个地方都要括号
- #define RADTODEG(x) ((x) * 57.29578)

带参数的宏



- 可以带多个参数
  - #define MIN(a,b) ((a)>(b)?(b):(a))
- 也可以组合（嵌套）使用其他宏

分号?



```
#define PRETTY_PRINT(msg) printf(msg);

if (n < 10)
    PRETTY_PRINT("n is less than 10");
else
    PRETTY_PRINT("n is at least 10");
```

带参数的宏



- 在大型程序的代码中使用非常普遍
- 可以非常复杂，如“产生”函数
  - 在#和##这两个运算符的帮助下
- 存在中西方文化差异
- 部分宏会被inline函数替代

其他编译预处理指令



- 条件编译
- error
- …

83头文件

头文件



- 把函数原型放到一个头文件（以.h结尾）中，在需要调用这个函数的源代码文件（.c文件）中#include这个头文件，就能让编译器在编译的时候知道函数的原型

#include



- #include是一个编译预处理指令，和宏一样，在编译之前就处理了
- 它把那个文件的全部文本内容原封不动地插入到它所在的地方
  - 所以也不是一定要在.c文件的最前面#include

“”还是<>



- #include有两种形式来指出要插入的文件
  - “”要求编译器首先在当前目录（.c文件所在的目录）寻找这个文件，如果没有，到编译器指定的目录去找
  - <>让编译器只在指定的目录去找
- 编译器自己知道自己的标准库的头文件在哪里
- 环境变量和编译器命令行参数也可以指定寻找头文件的目录

#include的误区



- #include不是用来引入库的
- stdio.h里只有printf的原型，printf的代码在另外的地方，某个.lib(Windows)或.a(Unix)中
- 现在的C语言编译器默认会引入所有的标准库
- #include <stdio.h>只是为了让编译器知道printf函数的原型，保证你调用时给出的参数值是正确的类型

头文件



- 在使用和定义这个函数的地方都应该#include这个头文件
- 一般的做法就是任何.c都有对应的同名的.h，把所有对外公开的函数的原型和全局变量的声明都放进去

不对外公开的函数



- 在函数前面加上static就使得它成为只能在所在的编译单元中被使用的函数
- 在全局变量前面加上static就使得它成为只能在所在的编译单元中被使用的全局变量

84声明

1.变量的声明:
- int i;是变量的定义
- externinti;是变量的声明

2.声明和定义
- 声明是不产生代码的东西
- 函数原型
- 变量声明
- 结构声明

- 宏声明
- 枚举声明
- 类型声明
- inline函数
- 定义是产生代码的东西声明和定义

3.头文件:

- 只有声明可以被放在头文件中
- 是规则不是法律
- 否则会造成一个项目中多个编译单元里有重名的实体
- * 某些编译器允许几个编译单元中存在同名的函数，或者用weak修饰符来强调这种存在

4.重复声明

- 同一个编译单元里，同名的结构不能被重复声明
- 如果你的头文件里有结构的声明，很难这个头文件不会在一个编译单元里被#include多次
- 所以需要“标准头文件结构”

5.

